v1.0.2
20260516

Programmazione Concorrente e Distribuita

Ingegneria e Scienze Informatiche - UNIBO

a.a 2025/2026

Prof. Alessandro Ricci

[Lab Notes]
MESSAGE-ORIENTED MIDDLEWARE (MOM)

REFERENCES

- *Message-Oriented Middleware.* Edward Curry. National University of Ireland, Galway, Ireland. Chapter 1 of “Middleware for Communications”. Wiley

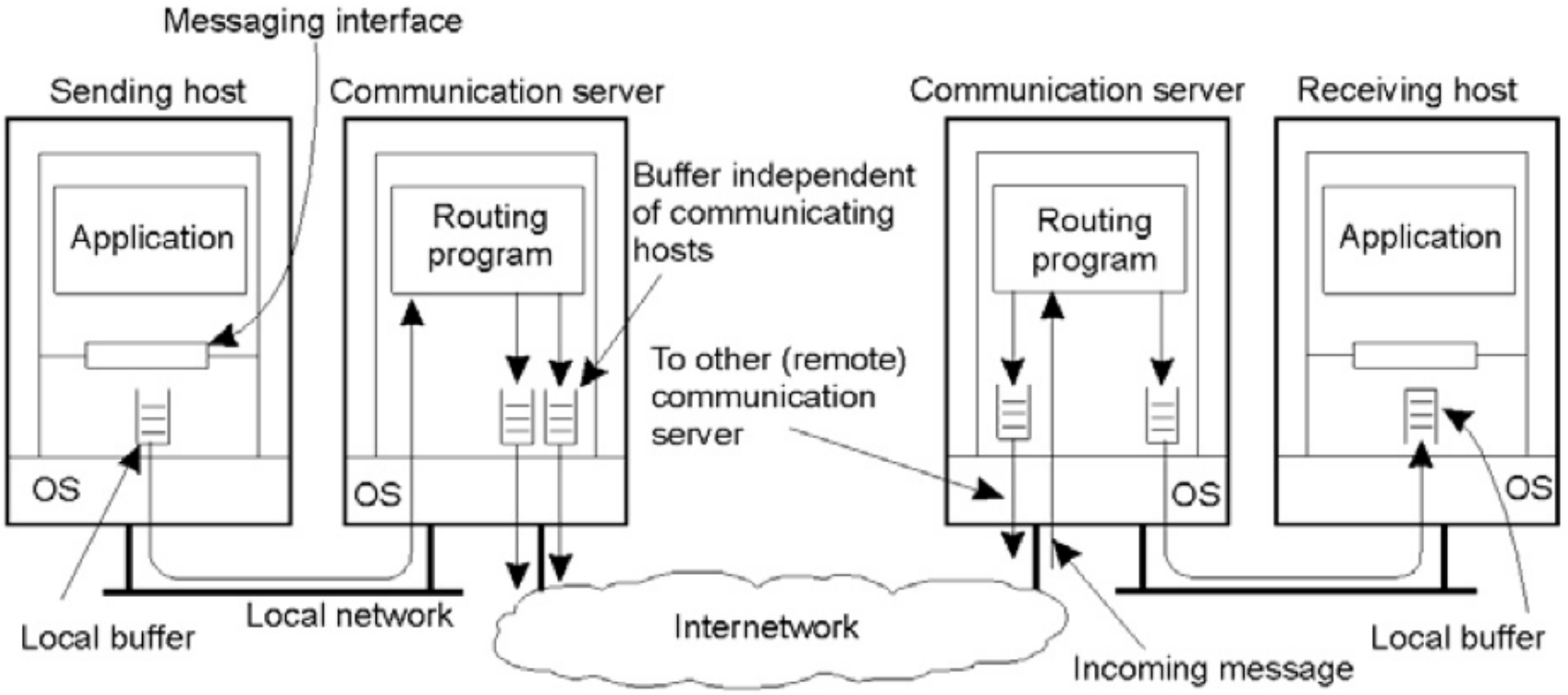
INTRODUCTION

- Need of messaging services enabling the flexible communication among different applications
  - with some QoS related to reliability, security, performance

=> Message-Oriented Middlewares (MOM)

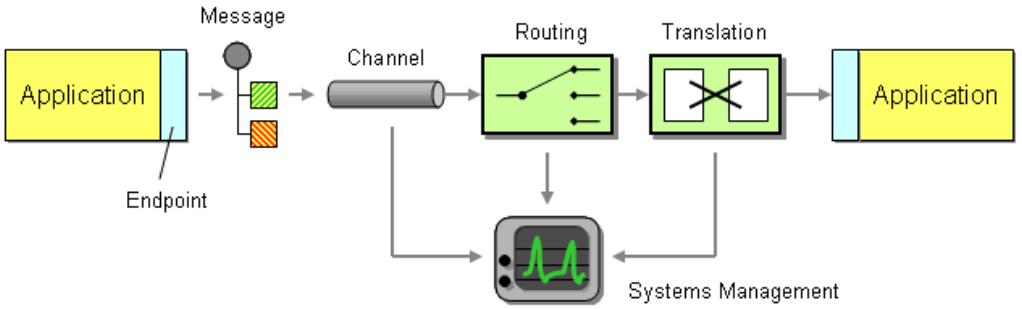
  - any middleware infrastructure that provides messaging capabilities
  - one of the cornerstone foundations of modern distributed enterprise systems
- A client of a MOM system can send messages to / receive messages from other clients of the messaging system
  - both centralised and distributed implementations

MOM ARCHITECTURE AT A GLANCE



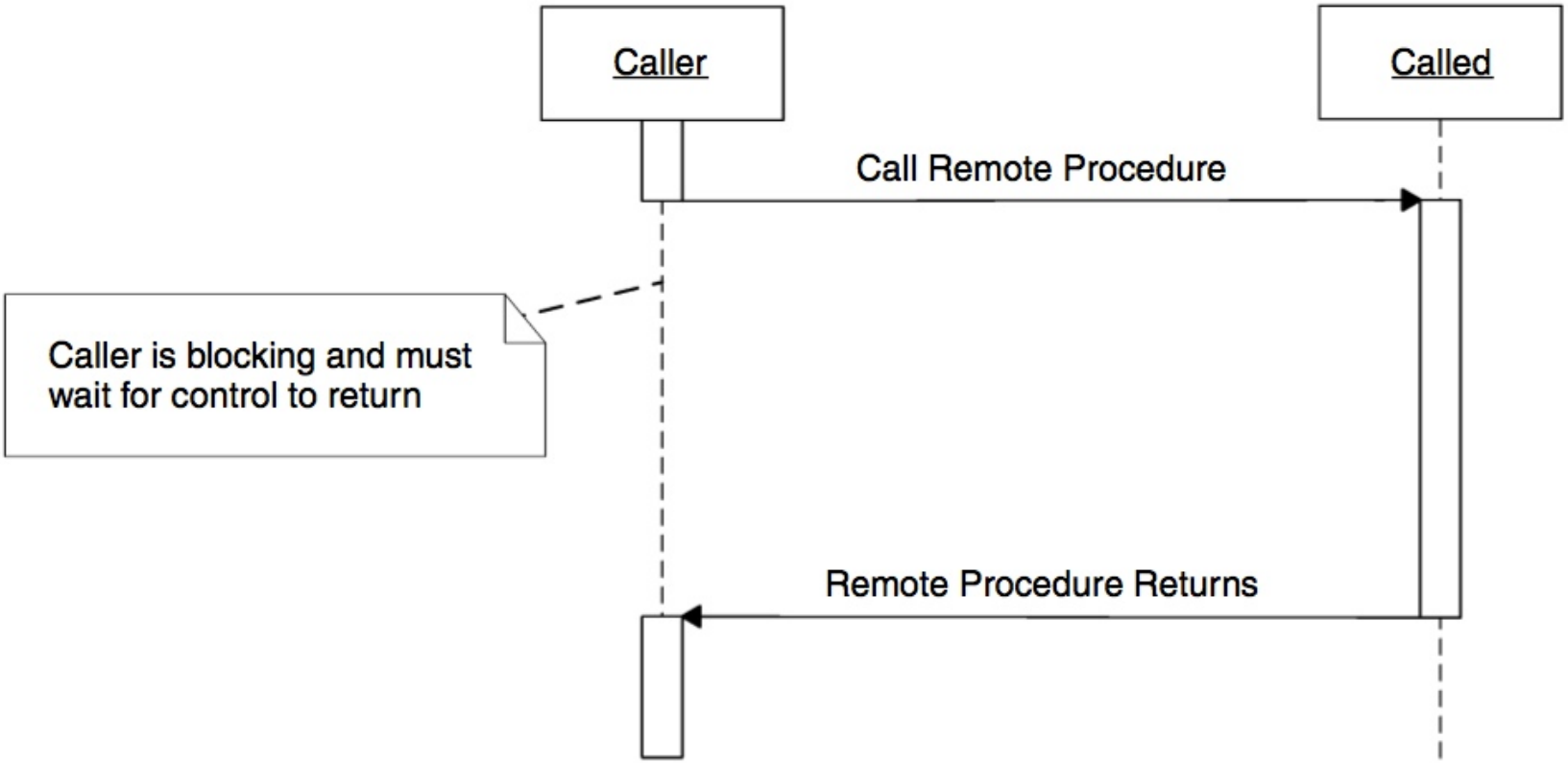
BASIC MESSAGING CONCEPTS

- *Channels*
  - medium to communicate
- *Messages*
  - piece of data exchanged
- *Pipes and Filters*
  - processing stages of messages
- *Routing*
  - channels composability
- *Transformation*
  - translating messages for adjusting heterogeneous data formats
- *Endpoint*
  - connecting an app to a messaging channel



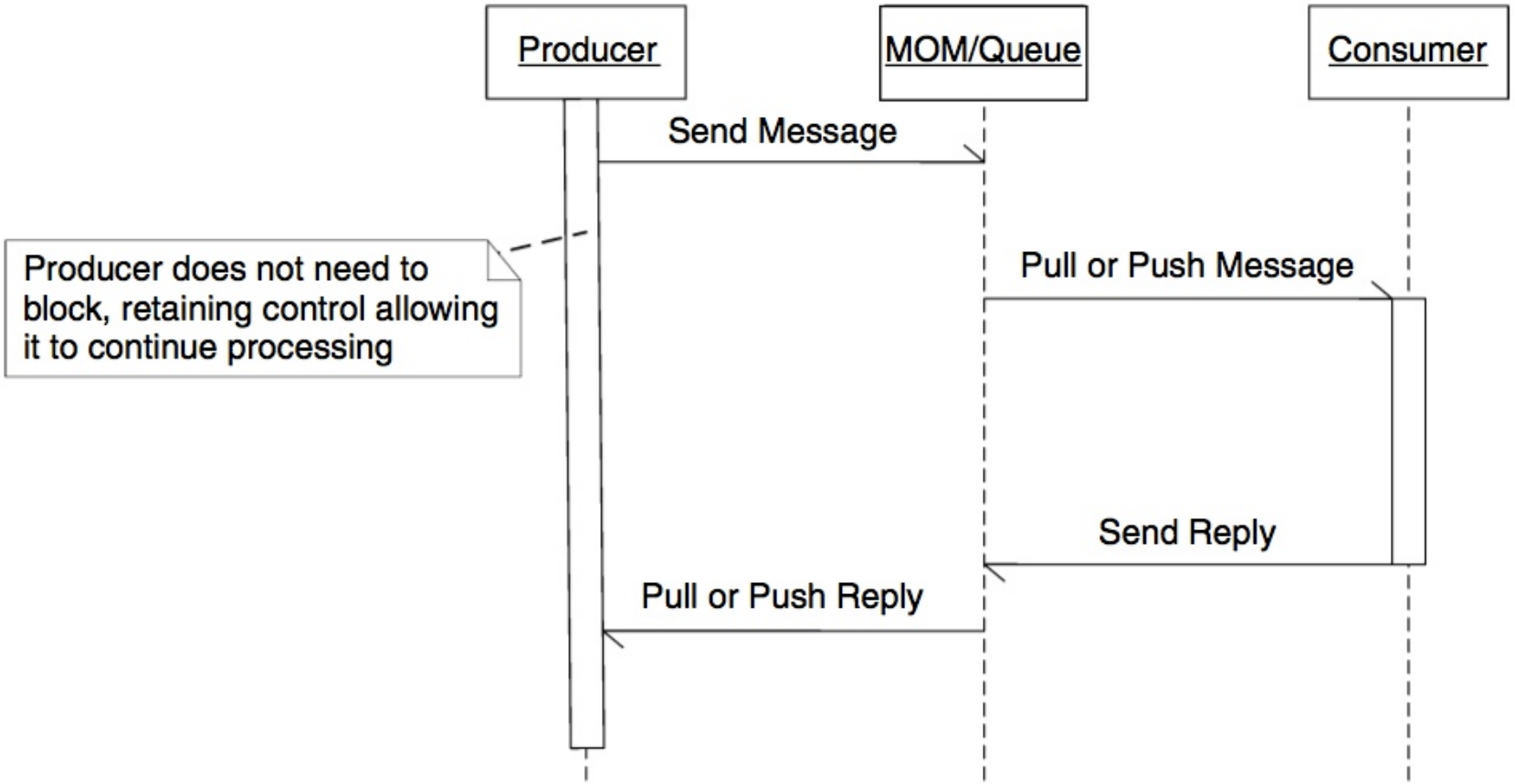
INTERACTION MODELS

- Synchronous interaction model



INTERACTION MODELS

- Asynchronous interaction model



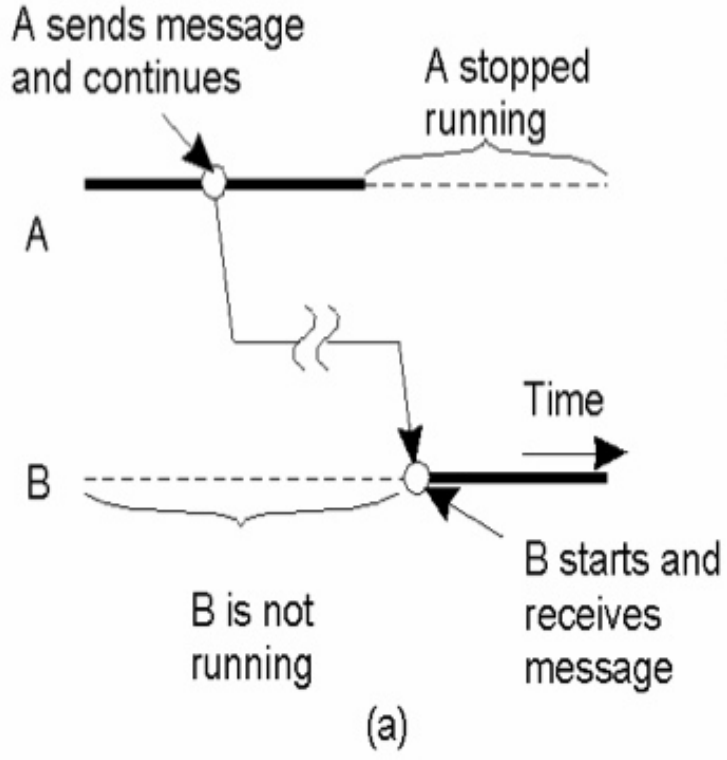
TRANSIENT VS. PERSISTENT

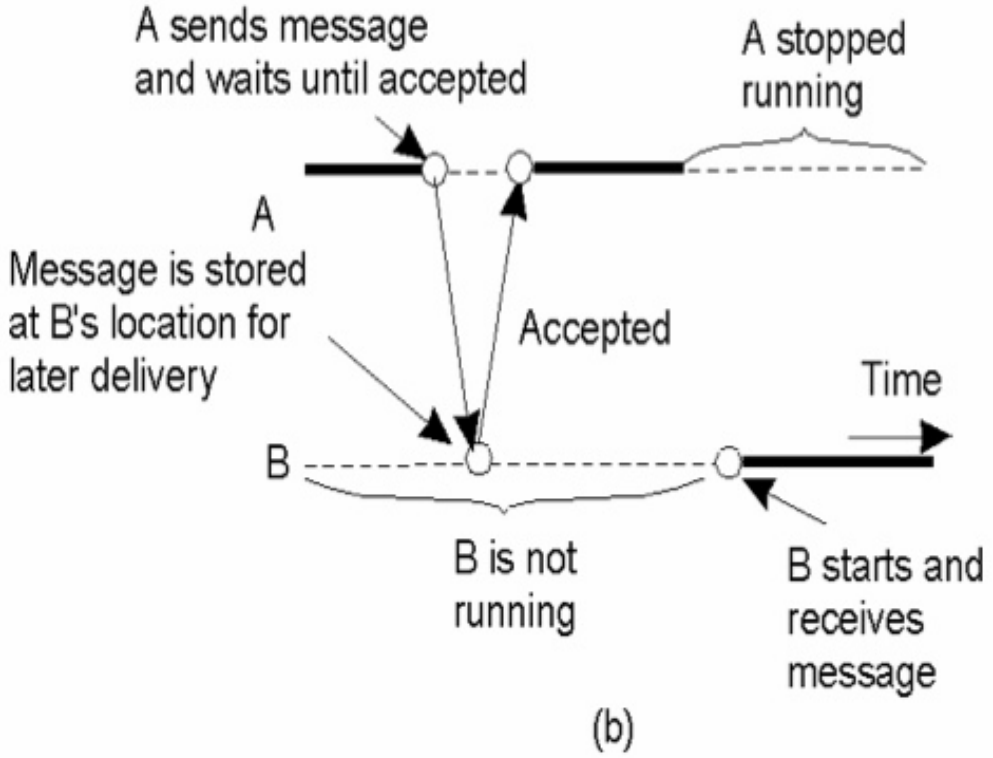
- Transient
  - supporting the communication of processes executing **at the same time**
    - sockets, RPC, RMI/CORBA, MPI (Message-Passing Interface)
  - not suitable for middleware that integrates applications in widely dispersed and large-scale distributed systems
- Persistent (=> MOM)
  - supporting the communication of processes **that are not necessarily executing at the same time**
    - message **queues** and **brokers**
  - this is the choice in large distributed heterogeneous systems
    - specifying policy about message delivery, message priorities, logging facilities, load balancing, fault tolerance, etc.

INTERFACE IN PERSISTENT MODELS

- Put
  - append a message to a specified queue
- Get
  - Block until the specified queue is nonempty, and remove the first message
- Poll
  - Check a specified queue for messages, and remove the first. Never block.
- Notify
  - Install a handler to be called when a message is put into the specified queue

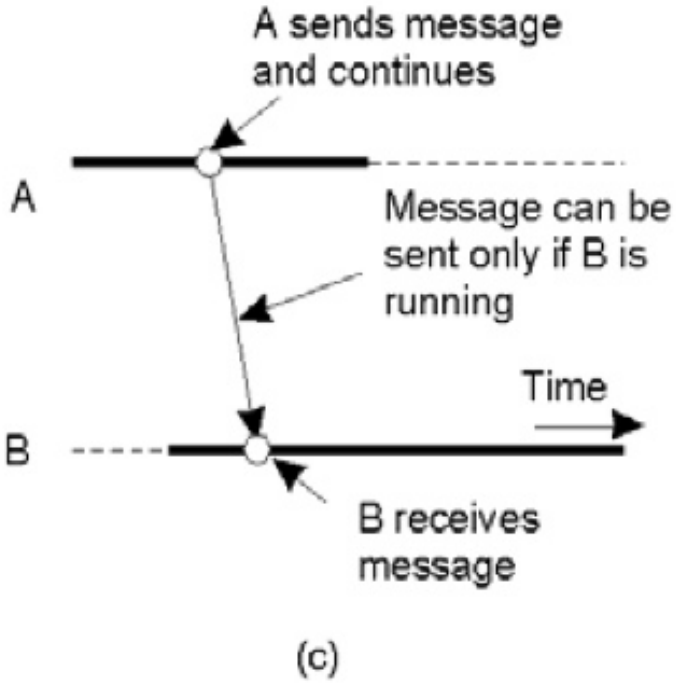
PERSISTENCE AND SYNCHRONICITY



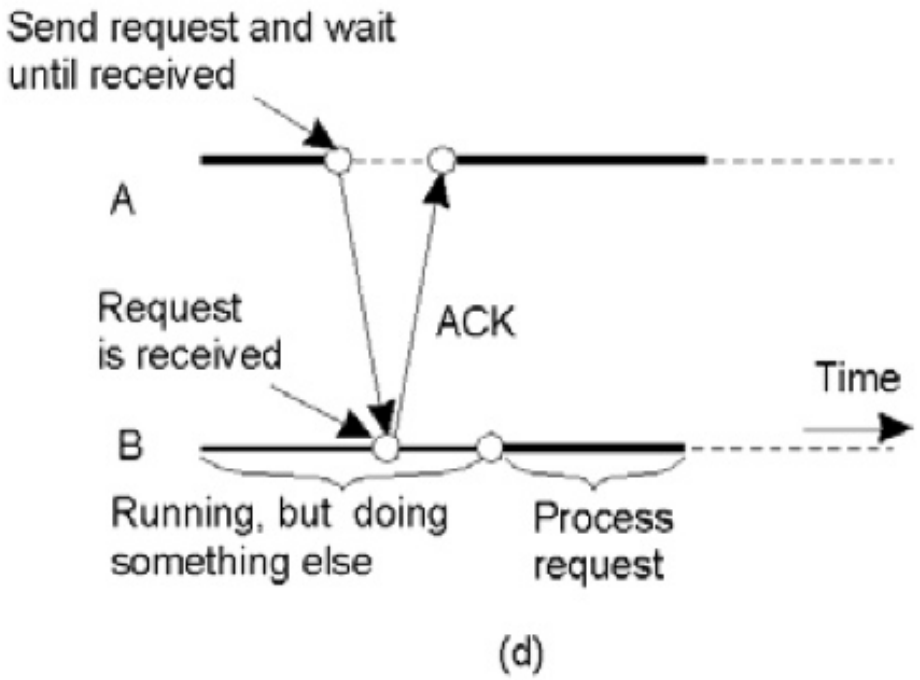


- (a) persistent, asynchronous communication
- (b) persistent, synchronous

PERSISTENCE AND SYNCHRONICITY



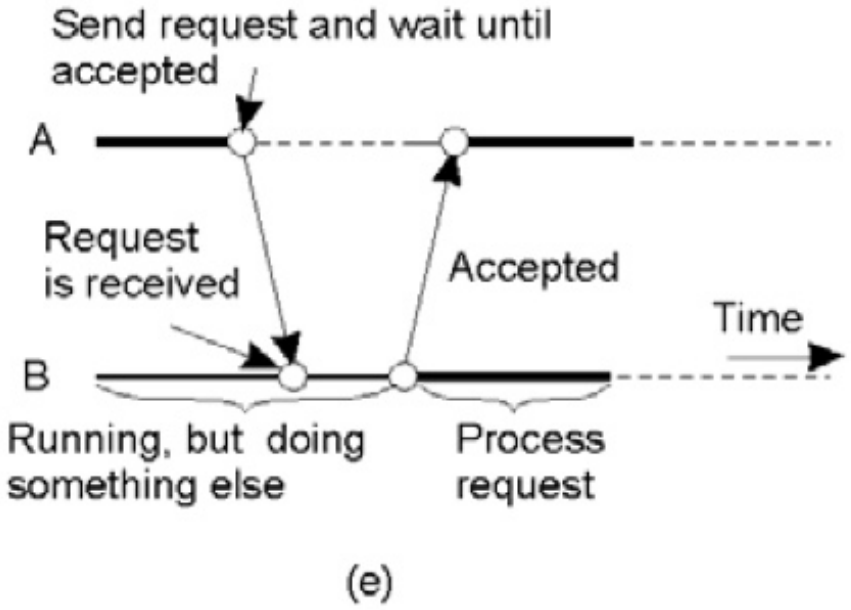
(c)



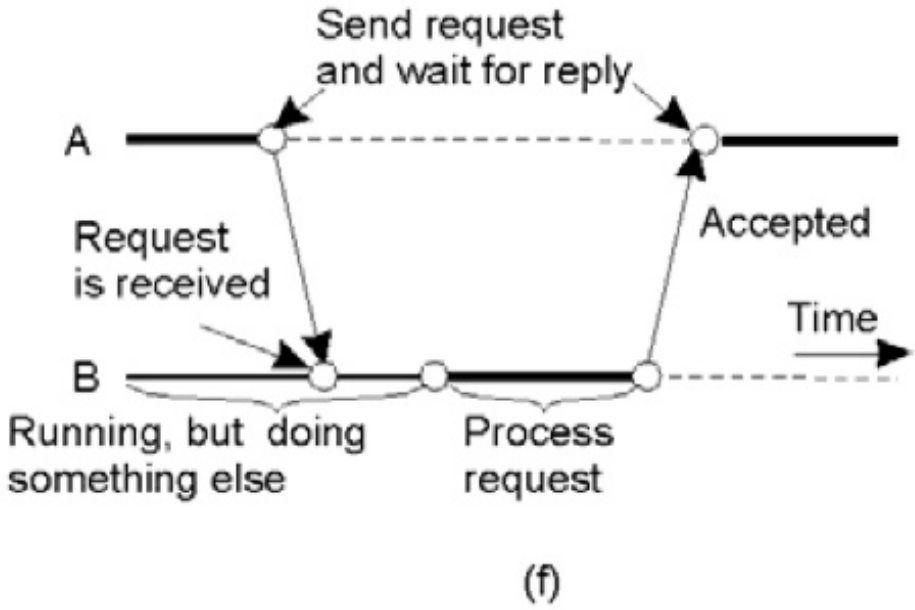
(d)

- (c) Transient, asynchronous
- (d) Receipt-based transient, synchronous

PERSISTENCE AND SYNCHRONICITY



(e)



(f)

- (e) Delivery-based transient synchronous communication at message delivery
- (f) Response-based transient synchronous communication

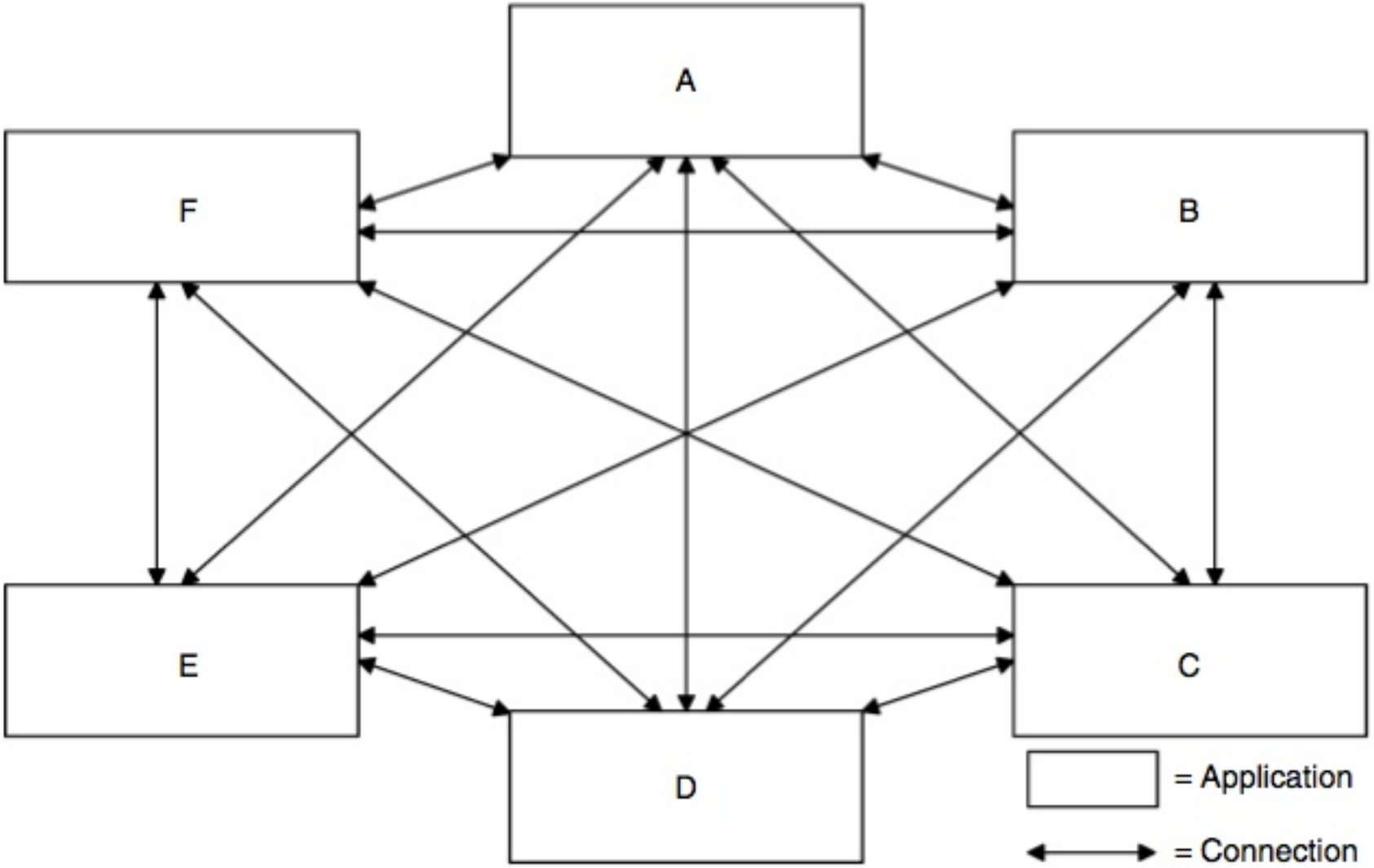
TRADITIONAL RPC

- Fundamental concept of distributed computing.
  - used in middleware platforms including CORBA, Java RMI, Microsoft DCOM, and XML-RPC.
- The objective of RPC is to allow two processes to interact, following the procedure-call synchronous model
  - RPC creates the facade of making both processes believe they are in the same process space (i.e., are the one process).
  - On the basis of the synchronous interaction model, RPC is similar to a *local procedure call*
    - whereby control is passed to the called procedure in a sequential synchronous manner while the calling procedure blocks waiting for a reply to its call.

TRADITIONAL RPC - PROBLEMS

- *Tight coupling*
  - any change to to interfaces must be propagated to every system
- *Reliability*
  - the synchronous model makes it hard to support a certain degree of reliability when failures occur
- *Scalability*
  - blocking nature can affect performances
- *Availability*
  - transient model => requires the simultaneous availability of participants => fragile to failures

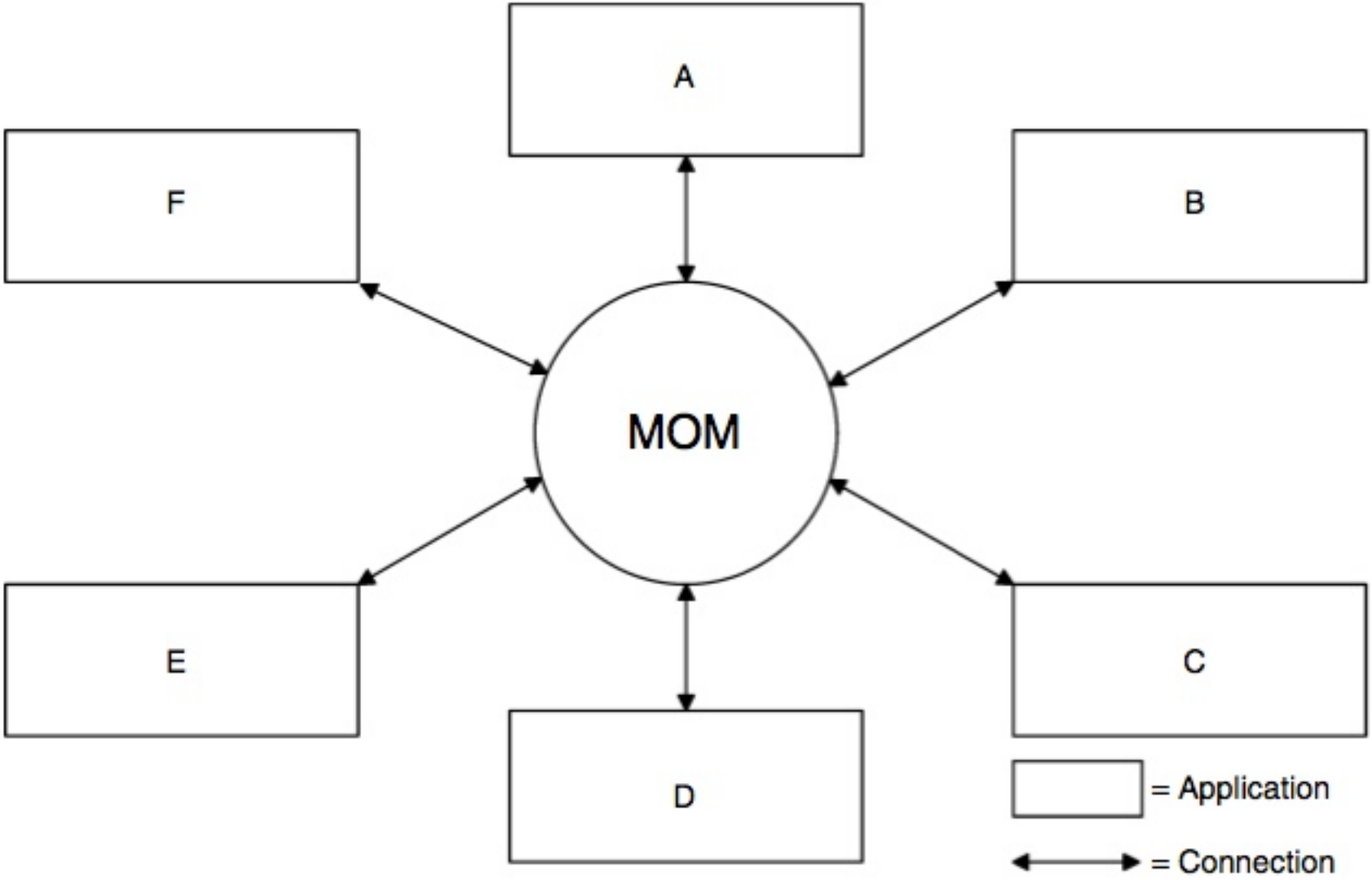
TIGHT COUPLING



FROM RPC TO MOM

- Message-Oriented communication model
  - making messages explicit
  - asynchronous interaction model
    - nonblocking
  - loose coupling & persistence
- MOM Middleware
  - message management, persistence and QoS
  - middle layer decoupling participants

MOM LOOSE COUPLING

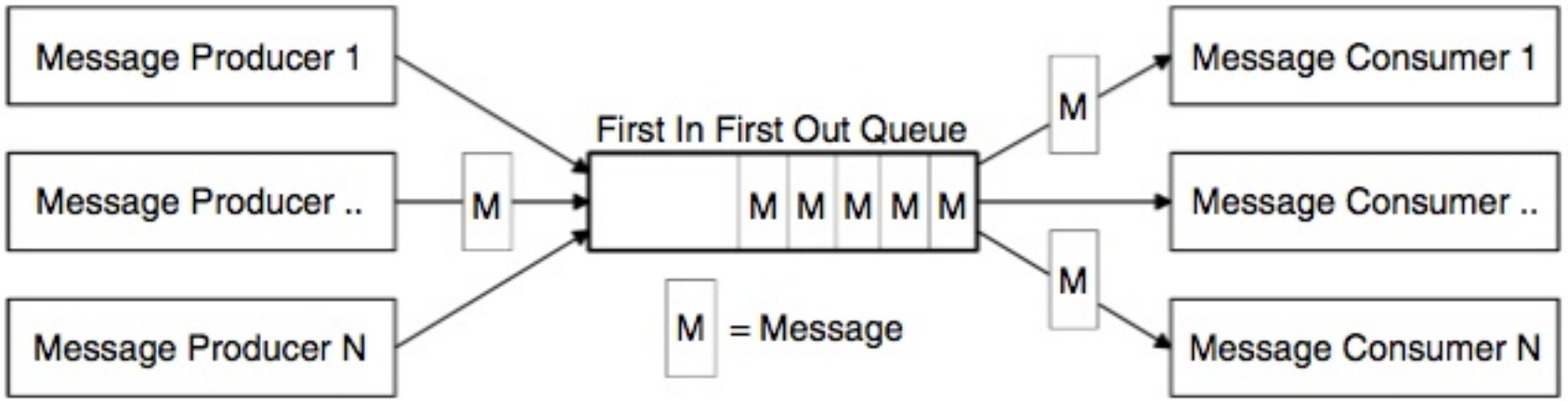


MOM BENEFITS

- Benefits
  - **reliability**
    - improved by message persistence typically implemented in MOM nodes
  - **scalability**
    - decoupling performance characteristics of the subsystems, which can be scaled independently
    - coping with unpredictable spikes in activities of subsystems
    - load balancing
  - **availability**
    - introducing high-availability allowing for continuous operation and smoother handling of system outages
- Drawbacks
  - complexity of asynchronous interactions management
  - coordination based on message passing
    - the users are responsible of ensuring correct order of events

MESSAGE QUEUE

- Fundamental concept in MOM
- Queues provide the ability to store messages on a MOM platform

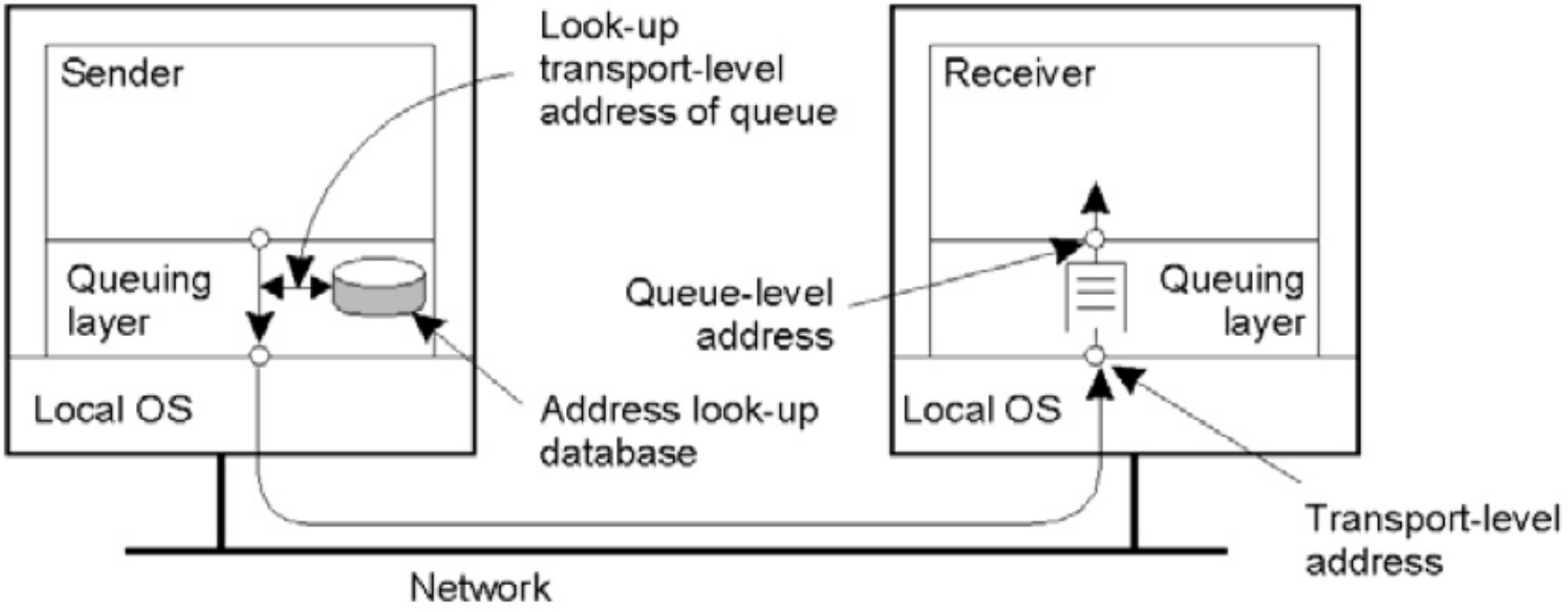


MESSAGE QUEUE TYPES

- Different kinds of queue type

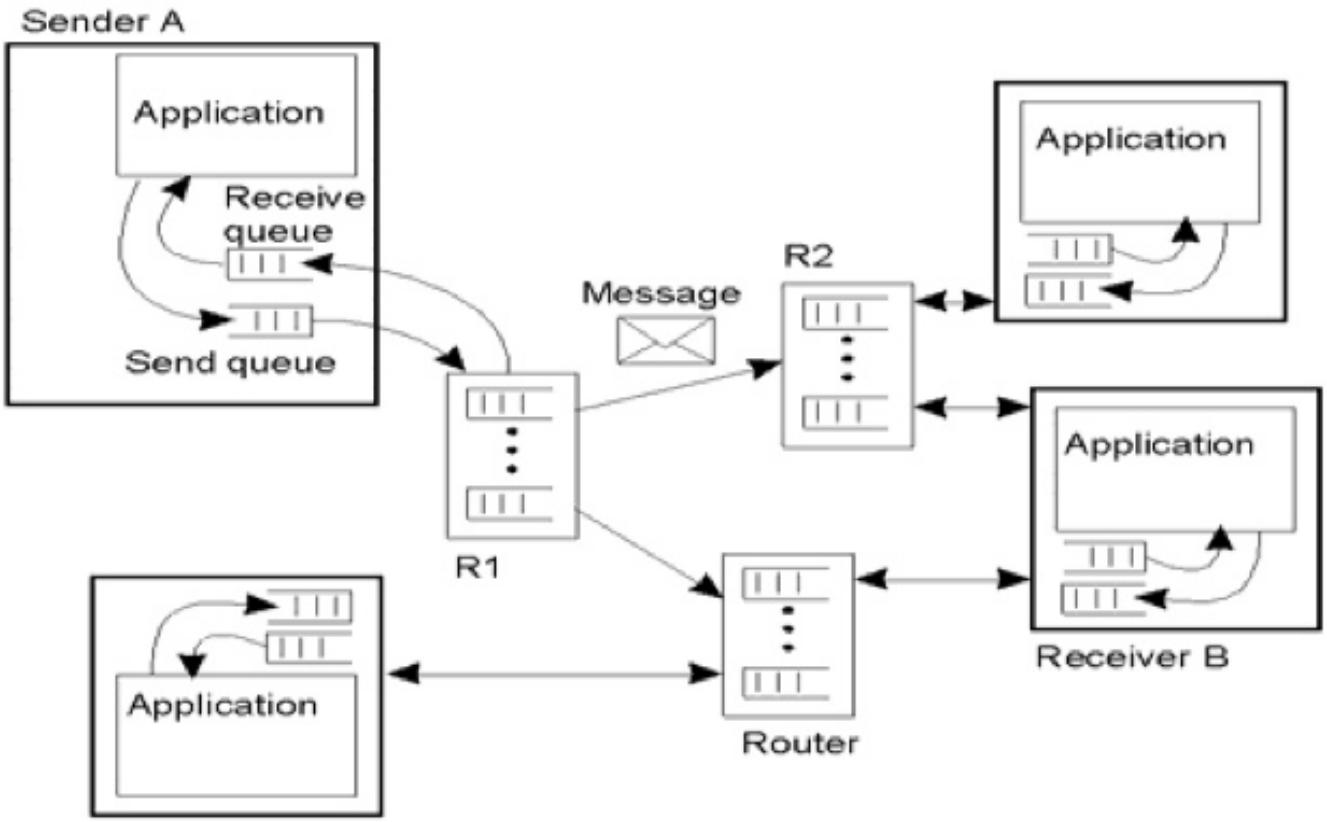
Queue type	Purpose
Public Queue	Public open access queue
Private Queue	Require clients to provide a valid username and password for authentication and authorization
Temporary Queue	Queue created for a finite period, this type of queue will last only for the duration of a particular condition or a set time period
Journal Queues	Designed to keep a record of messages or events. These queues maintain a copy of every message placed within them, effectively creating a journal of messages
Connector/Bridge Queue	Enables proprietary MOM implementation to interoperate by mimicking the role of a proxy to an external MOM provider. A bridge handles the translation of message formats between different MOM providers, allowing a client of one provider to access the queues/messages of another
Dead-Letter/Dead-Message Queue	Messages that have expired or are undeliverable (i.e., invalid queue name or undeliverable addresses) are placed in this queue

GENERAL ARCHITECTURE OF MOM WITH PERSISTENCE



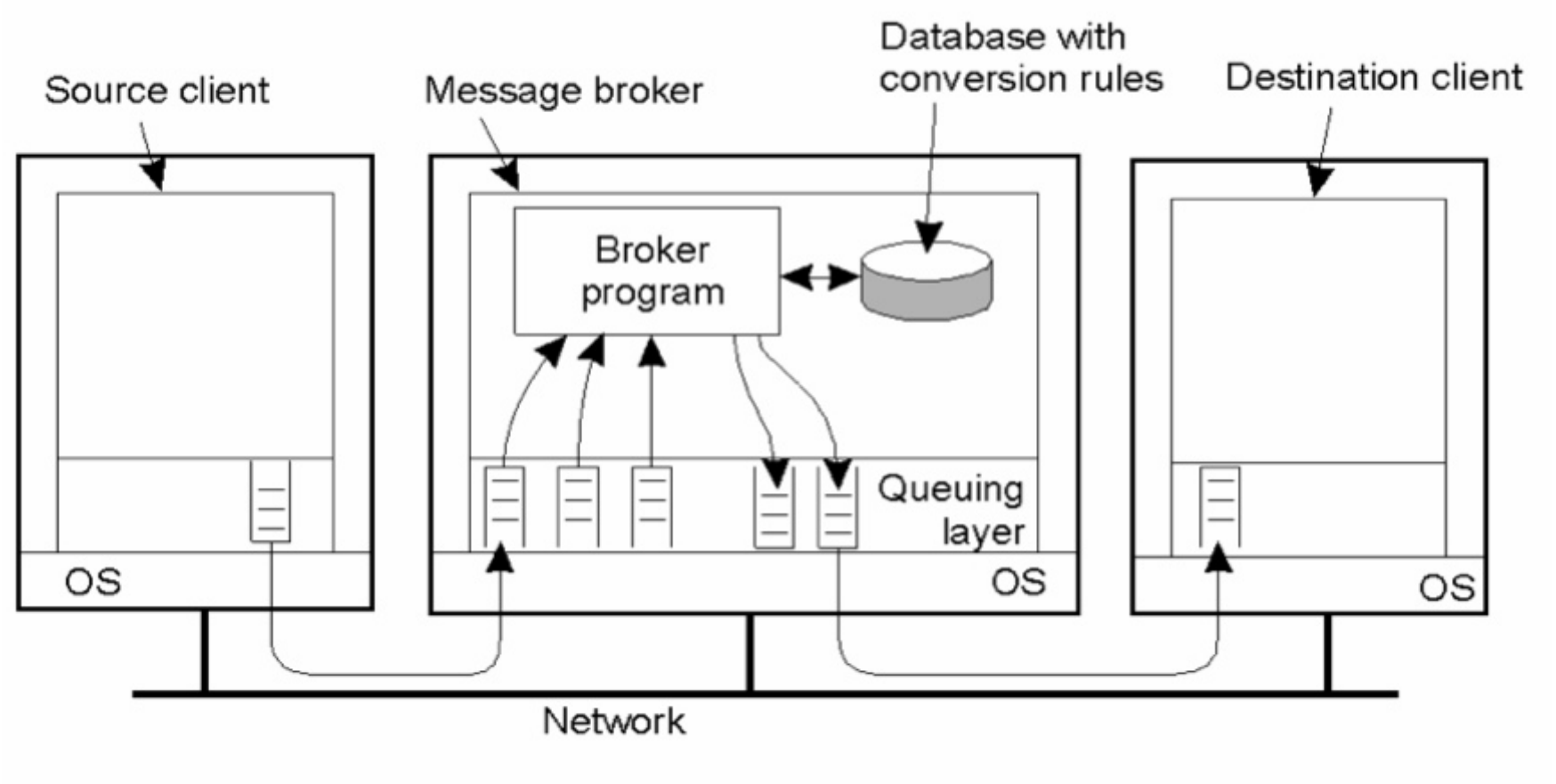
GENERAL ARCHITECTURE OF MOM WITH PERSISTENCE

- With routers



GENERAL ARCHITECTURE OF MOM WITH PERSISTENCE

- With brokers

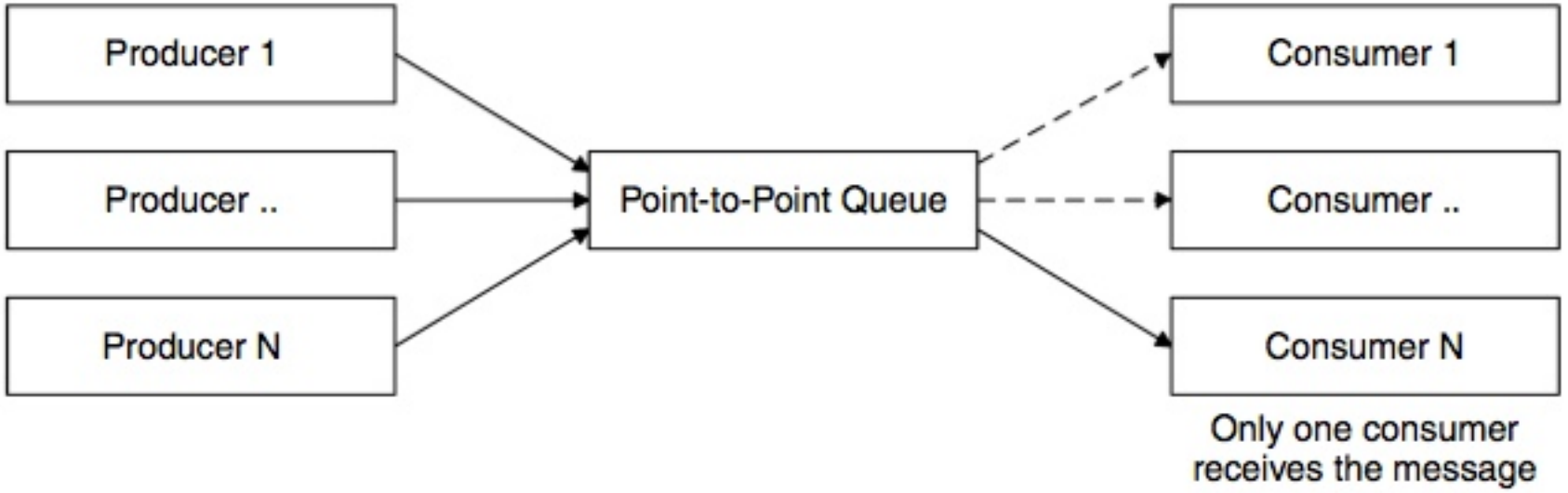


MESSAGING MODELS

- Point-to-Point
- Publish/Subscribe

POINT-TO-POINT

- Pure asynchronous message passing between software entities
  - typically more clients publishing a message in a queue and single consuming clients
  - FIFO queues



PUBLISH-SUBSCRIBE

- To disseminate information between anonymous message consumers and producers
  - one-to-many and many-to-many
- Two roles
  - publishers as clients publishing messages to a specific topic/channel
  - subscribers as clients subscribing topics wishing to consume messages, possibly specifying filters
  - various kinds of P/S systems with different features and techniques (filtering,…)

PUSH AND PULL

- A consuming client has two methods of receiving messages from the MOM provider
  - **pull**
    - a consumer can poll the provider to check for any messages, effectively pulling them from the provider
  - **push**
    - alternatively, a consumer can request the provider to send on relevant messages as soon as the provider receives them
    - they instruct the provider to push messages to them

MESSAGE FILTERING

- MOM feature that allows message consumer/receiver to be selective about the messages it receives from a channel

Filter type	Description
Channel-based	Channel-based systems categorize events into predefined groups. Consumers subscribe to the groups of interest and receive all messages sent to the groups
Subject-based	Messages are enhanced with a tag describing their subject. Subscribers can declare their interests in these subjects flexibly by using a string pattern match on the subject, for example, all messages with a subject starting of "Car for Sale"
Content-based	As an attempt to overcome the limitations on subscription declarations, content-based filtering allows subscribers to use flexible querying languages in order to declare their interests with respect to the contents of the messages. For example, such a query could be giving the price of stock 'SUN' when the volume is over 10,000. Such a query in SQL-92 would be "stock_symbol = 'SUN' AND stock_volume >10,000"
Content-based with Patterns (Composite Events)	Content-based filtering with patterns, also known as *composite events* [10], enhances content-based filtering with additional functionality for expressing user interests across multiple messages. Such a query could be giving the price of stock 'SUN' when the price of stock 'Microsoft' is less than $50

HIERARCHICAL CHANNEL NAMESPACES

- Grouping mechanism in pub/sub messaging model.
  - channels structured in a hierarchical fashion, so that they may be nested under other channels.
  - each subchannel offers a more granular selection of the messages contained in its parent.
- Clients of hierarchical channels subscribe to the most appropriate level of channel in order to receive the most relevant messages.
- Important mechanisms helping to manage large volumes of different messages

MOM QUALITY OF SERVICE

- MOM can provide different forms of quality of services
- Common one is about message delivery guarantees (in the case of failures)
  - **at-least-once**
  - **at-most-once**
  - **exactly-once**

MOM IMPLEMENTATIONS AND STANDARDS

- Large number of implementations exist
  - WebSphere MQ (formerly MQSeries), TIBCO, SonicMQ, Herald, Hermes, SIENA, Gryphon, JEDI, REBECCA and OpenJMS
- Proposed Standards
  - **Advanced Message Queuing Protocol (AMQP)**
    - OASIS standard that defines the protocol and formats used between participating application components, so implementations are interoperable.
  - The **eXtensible Messaging and Presence Protocol (XMPP)**
    - a communications protocol for message-oriented middleware based on XML (Extensible Markup Language).
  - Java Message Service (JMS)
  - the CORBA Event Service, CORBA Notification Service

OPEN SOURCE IMPLEMENTATIONS

- Apache **ActiveMQ**
  - http://activemq.apache.org
  - message broker which supports multiple wire level protocols for maximum interoperability
    - AMQP, MQTT, OpenWire, REST, RSS and Atom, Stomp, WSIF, WS Notification, XMPP
- **RabbitMQ**
  - http://www.rabbitmq.com/
- **MQTT**
  - for IoT contexts
- …

FROM MESSAGES TO EVENTS AND EVENT BROKERS

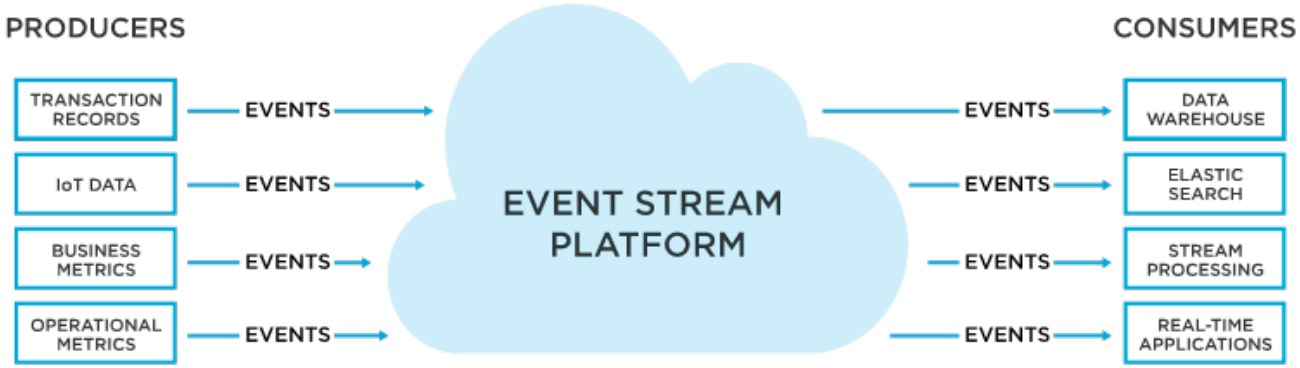
- Middleware/platforms specialised in managing **events** and supporting event-driven interactions
- “Events”
  - an event can be anything that has happened within the scope of the business communication structure, anything important to the business
    - e.g. receiving an invoice, booking a meeting room, hiring a new employee,..
  - an event is a recording of what happened
  - once events start being captured, event-driven systems can be created to harness and use them across the organization
- **Event brokers** enable an immutable, append-only log of facts that preserves the state of event ordering.
  - consumers can pick up and reprocess from anywhere in the log at any time
  - *single source of truth* for all applications

EVENT BROKERS VS. MESSAGE BROKERS

- Event brokers can be used in place of a message broker, but a message broker cannot fulfill all the functions of an event broker.
  - message brokers (in MOM) enable systems to communicate across a network through publish/subscribe message queues
    - producers write messages to a queue, while a consumer consumes these messages and processes them accordingly.
    - messages are then acknowledged as consumed and deleted either immediately or shortly thereafter
- Event brokers are designed around *providing an ordered log of facts.*
  - unlike the message broker, the event broker maintains a single ledger of records and manages individual access via indices, so each independent consumer can access all required events
  - a message broker deletes events after acknowledgment, whereas an event broker retains them for as long as the organization needs.
    - the deletion of the event after consumption makes a message broker insufficient for providing the indefinitely stored, globally accessible, replayable, single source of truth for all applications.

DISTRIBUTED EVENT STREAMING PLATFORMS

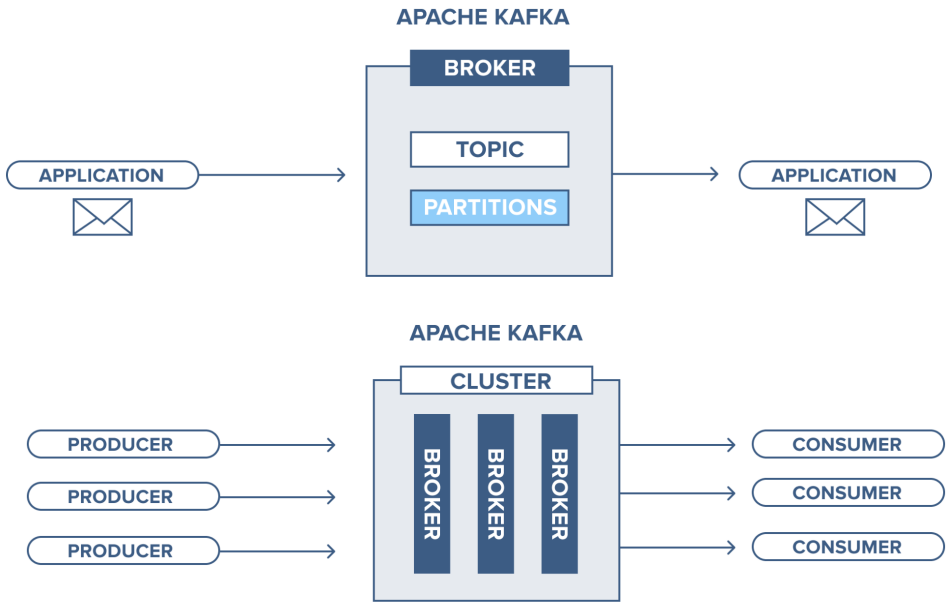
- Event brokers are the core of modern **distributed event streaming platforms**



- Important element of modern data-driven architectures
  - both batch + real-time data analysis

APACHE KAFKA

- Example of distributed event stream platform (open-source): **Apache Kafka**
  - https://kafka.apache.org/
  - based on publish/subscribe messaging
- Main concepts: **topics**, managed by *brokers*, organised in *clusters*
  - high scalability, low latency



(images from https://www.cloudkarafka.com/blog/part1-kafka-for-beginners-what-is-apache-kafka.html)